

Module 29: Abstract Classes, Interfaces & Advanced Polymorphism

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Module 12 previewed abstract classes; since then students have used `= 0` in several starters. This module completes the story. Students learn why a pure virtual function is better than a misleading default, build an abstract class that shares a procedure while subclasses supply its steps (the Template Method idea), and design interfaces - pure contracts with no data - that the rest of the program depends on. They look underneath at the vtable, distinguish static from dynamic binding, use `dynamic_cast` and `typeid` while learning why needing them is often a smell, copy polymorphic objects with `clone()`, and avoid the trap of virtual calls in constructors.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Write pure virtual functions and abstract classes, and explain why a pure virtual beats a default implementation.
- Build an abstract class whose non-virtual method runs a fixed procedure using protected pure virtual steps.
- Design interfaces and write functions that depend only on an interface.
- Explain how vtables implement dynamic dispatch, and distinguish static from dynamic binding.
- Use `dynamic_cast` and `typeid`, and recognise when a virtual function should replace them.
- Copy polymorphic objects with a virtual `clone()`, and recognise covariant return types.
- Explain why virtual calls in constructors and destructors don't reach derived overrides.

Prerequisites

- Module 28 (advanced inheritance).
- Module 26 (`unique_ptr`, used for `clone()`).

Materials & Instructor Prep

- 29-abstract-classes-interfaces/examples/ 01 to 06; 02_interfaces.cpp and 04_dynamic_cast_typeid.cpp are central.
- A vtable diagram prepared on the board or a slide.

Key Vocabulary

Term	Definition
Pure virtual function	A virtual function declared = 0 with no body: every concrete derived class must provide it.
Abstract class	A class with at least one pure virtual function; it cannot be instantiated, only derived from.
Interface	A class with only pure virtual functions and a virtual destructor, and no data: a pure capability contract.
Template Method	A non-virtual method that runs a fixed sequence of steps, some supplied by derived classes through virtual functions.
vtable / vptr	A per-class table of virtual function addresses, and the hidden pointer in each object that points to its class's table.
Static vs dynamic binding	Choosing the function at compile time from the declared type, vs at runtime from the actual object type.
RTTI	Run-Time Type Information: <code>dynamic_cast</code> and <code>typeid</code> , which query an object's actual type at runtime.
<code>clone()</code>	A virtual method that returns a copy of the object as its real type, used for polymorphic copying.

Lesson Outline & Instructor Talking Points

1. Pure virtual functions and abstract classes (15 min)

- Contrast Module 12's 'default returns 0' with = 0: one compiles a silent lie, the other makes forgetting a compile error.
- Run examples/01_abstract_class.cpp. Uncomment `HalfFinishedSensor`'s object to show the error message naming the missing function.
- Highlight `report()` as the Template Method idea: the shared procedure, with protected steps supplied by each sensor.

2. Interfaces and programming to an interface (20 min)

- Run examples/02_interfaces.cpp. Point out that `saveAll()` never mentions `StudentRecord` or `SensorReading` - it depends only on `Saveable`.
- Ask: next year a new 'Invoice' class must be saved. What changes in `saveAll`? (Nothing.) Name this as the idea Module 33 formalises.

Instructor note: This is the most important design idea in the module. Give students one minute to name a capability-interface from their own track.

3. Under the hood: vtables and binding (15 min)

- Draw the vtable diagram from the README. Run examples/03_static_vs_dynamic_binding.cpp and use sizeof to show the hidden pointer.
- Show the non-virtual category() being statically bound - the classic Module 12 surprise, now with a name.

4. dynamic_cast, typeid, and the smell (15 min)

- Run examples/04_dynamic_cast_typeid.cpp: pointer form returns nullptr, reference form throws std::bad_cast.
- Put the type-checking chain next to the virtual area() loop and ask what happens when a Triangle is added to each.

5. clone() and the constructor trap (15 min)

- Run examples/05_clone_covariant.cpp: deep-copy a mixed collection, scale only the copies.
- Run examples/06_virtual_in_constructor.cpp after students predict the output; explain using Module 23's construction order.

Common Student Misconceptions

Misconception	How to Address It
"An abstract class can't have data or implemented methods."	It can have both. Only an interface is restricted to pure virtual functions; abstract classes often share real code.
"dynamic_cast is the normal way to handle different derived types."	Choosing behaviour by checking types is what virtual functions replace. Reserve dynamic_cast for genuine capability questions.
"Calling a virtual function in a base constructor calls the derived version."	During the base constructor the derived part doesn't exist yet, so the base version runs (or a pure virtual call crashes).
"Copying through a base reference copies the whole object."	It slices, and an abstract base can't be copied at all. Use a virtual clone() that knows the real type.

Exercises & Module Project

Exercise: 29-abstract-classes-interfaces/exercises/exercise1.cpp - a Payable interface, an abstract Employee with a Template Method payslip and clone(), salaried and hourly employees, a Supplier that is Payable but not an Employee, an interface-only payAll(), and a justified use of dynamic_cast. Project (three tracks): Generic - assessments and report cards; EE - a sensor framework; Biomedical - vital-sign monitoring. Each builds an abstract base with a Template Method, three concrete classes, two interfaces (one implemented outside the hierarchy), clone(), and a vtable diagram.

Facilitation guidance: Check that the functions taking interfaces never name a concrete class, that the Template Method is non-virtual with protected pure virtual steps, and that any dynamic_cast is accompanied by a convincing justification. Students who write 'my design never needed it' with a reason should get full credit.

Discussion & Comprehension Check Questions

1. Why is `virtual double area() const = 0;` better than a version that returns 0?

Guidance: With `= 0`, a derived class that forgets `area()` can't be instantiated - a compile error. A default of 0 compiles and silently reports wrong results.

2. What does it mean to 'program to an interface'?

Guidance: Writing code that depends only on an abstract capability, so it works unchanged with every current and future class that implements it.

3. Why can't you copy a polymorphic object through a `Shape&` with `Shape copy = s;`?

Guidance: It would slice the object to its `Shape` part - and an abstract `Shape` can't exist at all. `clone()` lets each class copy itself as its real type.

Assessment Ideas

- Students write an interface for a capability in their track and two unrelated classes that implement it.
- Show a `dynamic_cast` chain and ask students to refactor it into a virtual function, explaining what changes when a new type is added.

Answer Key Notes

See [29-abstract-classes-interfaces/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Our hierarchies are now well designed. Next module returns to value types - Money, Fractions, Impedance - and gives them a complete, natural set of operators."

Instructor Tips

- If time is short, protect interfaces (step 2) and the `dynamic_cast` smell (step 4); the vtable can be a five-minute diagram.
- Encourage students to name interfaces after capabilities (Saveable, Alarmable) rather than things.